

UX Design Plan — onboarding a new user into the governed loop

Working design direction, derived from the dual-track design research (archived at [archive/2026-07-12-dual-track-design-research.md](#)). Goal: a first-time dev user completes the full loop — add a source → ask → propose → review → rule → handoff → trace → learn — **without a guide**. This plans the next UX layer on top of the beginner-surface simplification already shipped.

The one idea that unlocks everything: it's Git for your team's knowledge

The hardest moment for a new user is **Step 3 — moving something from your private Personal Brain to the shared Company Brain ("propose to share")**. Research found this is structurally identical to Git's staging area (`git add` → `git commit`) — the exact spot where every Git beginner asks "why two steps?" The answer that makes it click is the same for both: **control**. You choose exactly what goes in.

Our buyers are developers. They already understand this. So we don't teach a novel concept — we say:

Artemis is Git for your team's knowledge and agent context. Your Personal Brain is your working copy. "Propose" is a commit. A reviewer merges it into the Company Brain. Agents only read what's been merged.

Use this framing everywhere — onboarding, docs, the pitch. It collapses the whole "why is this two steps / why can't I just share" confusion into one sentence a dev already believes.

Core principle: teach the model *before* the user acts in it

The research's clearest finding (from Linear, Figma, NotebookLM): don't drop users into an empty workspace and hope they infer the model. Teach the two-brain model *before* first capture, then drive to **one completed loop in the first session**.

The design interventions, in priority order

Each maps to a real risk moment. Build them in this order.

P0 — Step 3: the "why two steps" moment (highest risk)

- At the first proposal, show an **inline panel** (not a FAQ, not a tooltip that vanishes): "Your Personal Brain is yours alone. Nothing here is visible to your team until you propose it and a reviewer approves."

This keeps your private thinking private and your team's knowledge trustworthy." One sentence of **control + privacy**, at the friction point.

- **Always say "propose to share," never "share."** It's the load-bearing word.
- **Pre-fill the first proposal from the user's own private answer**, so the action is concrete — and **show a preview of exactly what the reviewer will see.**
- Pair with a small persistent diagram: `Personal` → [review] → `Company` .

P0 — The two-brain namespace (make it impossible to confuse)

- **Persistent, visually distinct layout** for Personal vs Company — different background/iconography, and a permanent **"Private — only you"** label on the Personal side. Not a toggle that hides one side; keep the relationship always visible.
- **Teach-the-model screen on first run** (skippable, re-openable): one interactive diagram — *"Everything starts in your Personal Brain (private). Reviewed knowledge lives in the Company Brain (shared). Agents read only the Company Brain."*
- **Never render the two brains with identical visual treatment.**

P1 — The empty state (first thing a real user sees)

- **Role-aware, instructional, one concrete first action.** For an engineer: *"Add the doc your last agent got wrong — a spec, a runbook, or a PR description."* Positive title, one primary action, state the payoff. Never a blank screen.
- **Seed a starter example** the user can inspect and safely delete (a sample "How we handle X" rule), so it's never truly empty.
- **Concierge tie-in:** because the founder is hand-seeded during concierge onboarding, the *founder* rarely sees an empty state — the **2nd-to-Nth users (engineers, new hires) do**. Design the empty state for *them*, and make sure concierge seeding means they arrive to an already-useful brain, not a void. (This is also the direct fix for the "slow time-to-value" risk.)

P1 — The privacy/trust answer, at first touch

- **Always-visible on the Personal Brain:** *"Only you can see this. Not your admin, not your team, not agents — until you propose it and a reviewer approves."* Users won't capture honestly into a Personal Brain they don't trust.
- A short **"How privacy works"** link that turns a hidden strength into visible trust: promotion is enforced server-side (can't be overridden by the client), and all five abuse categories (tenant leakage, prompt injection, indirect injection, memory poisoning, excessive agency) are tested and fail closed.

P2 — The reviewer's rubric (so approval isn't a blank stare)

- Inline **three-check rubric** in the approval view: **"Is it true? Is it reusable? Is it safe for agents to obey?"** — Linear-Triage style checklist.
- **Show provenance:** what source it came from, who proposed it, and the private answer it was drawn from.

- **The reviewer's decision writes a reason** that becomes part of the trace — reinforcing "trace = a governed record of what *and why*," not a log.

Vocabulary (lock these)

Use	Never	Why
"Propose to share" / "share proposal"	"share"	The two-step control model is the moat; the word carries it.
"Team rule" + "Rules are what your agents follow"	"document"	Signals agents obey it.
"Trace — what the agent did <i>and why</i> "	"log"	The "and why" is the differentiator.
"Personal Brain (Private — only you)"	"Personal Brain" alone	The privacy qualifier is load-bearing.

Information architecture (sidebar) — wedge-scoped

- Two clearly separated spaces: **Personal Brain (private)** and **Company Brain (shared)** — the primary split.
- Under each, the **loop objects** a beginner needs (sources, ask, proposals, team rules, work, trace) — not the full future suite.
- **Show the brain growing via a "What changed" feed / timeline** (new lessons, edits, decisions over time) — *not* a knowledge-graph visualization. This is the right answer to "let users see it grow": a legible history, like a commit log, not a spider-web.
- **Out of scope for this UX:** a full productivity suite, a "news" feed, company formation, BI. Don't re-add them to the sidebar.

What NOT to do

- **No graph theater.** A raw knowledge-graph is an anti-pattern for everyday users; it looks smart and helps no one. Growth = a what-changed timeline.
- **Don't default to shared.** Our beachhead users live in Notion/Google Docs and expect default-shared — actively counter-teach that. Private is the home base.
- **Don't bury the "why" in a FAQ.** Every friction point gets its one-sentence reason inline, at the moment of friction.

How we validate it

Run the WF-6 comprehension test with the **New Hire persona (zero context), not the Founder**. The bar: after one unaided loop, that person can say aloud — *what is this, what do I do first, and what stays private vs. goes to the team vs. goes to agents*. If a zero-context user can answer all three, the UX works.

Relationship to what's already built

The beginner-surface simplification is done (a beginner sees ~6 objects; advanced machinery is hidden, not renamed). **This plan is the next layer:** teach the model up front, nail Step 3 with the Git framing, make the two brains visually unmistakable, seed the empty state, and give the reviewer a rubric. None of it weakens the governance — it makes the governance *legible*, which is the whole point.